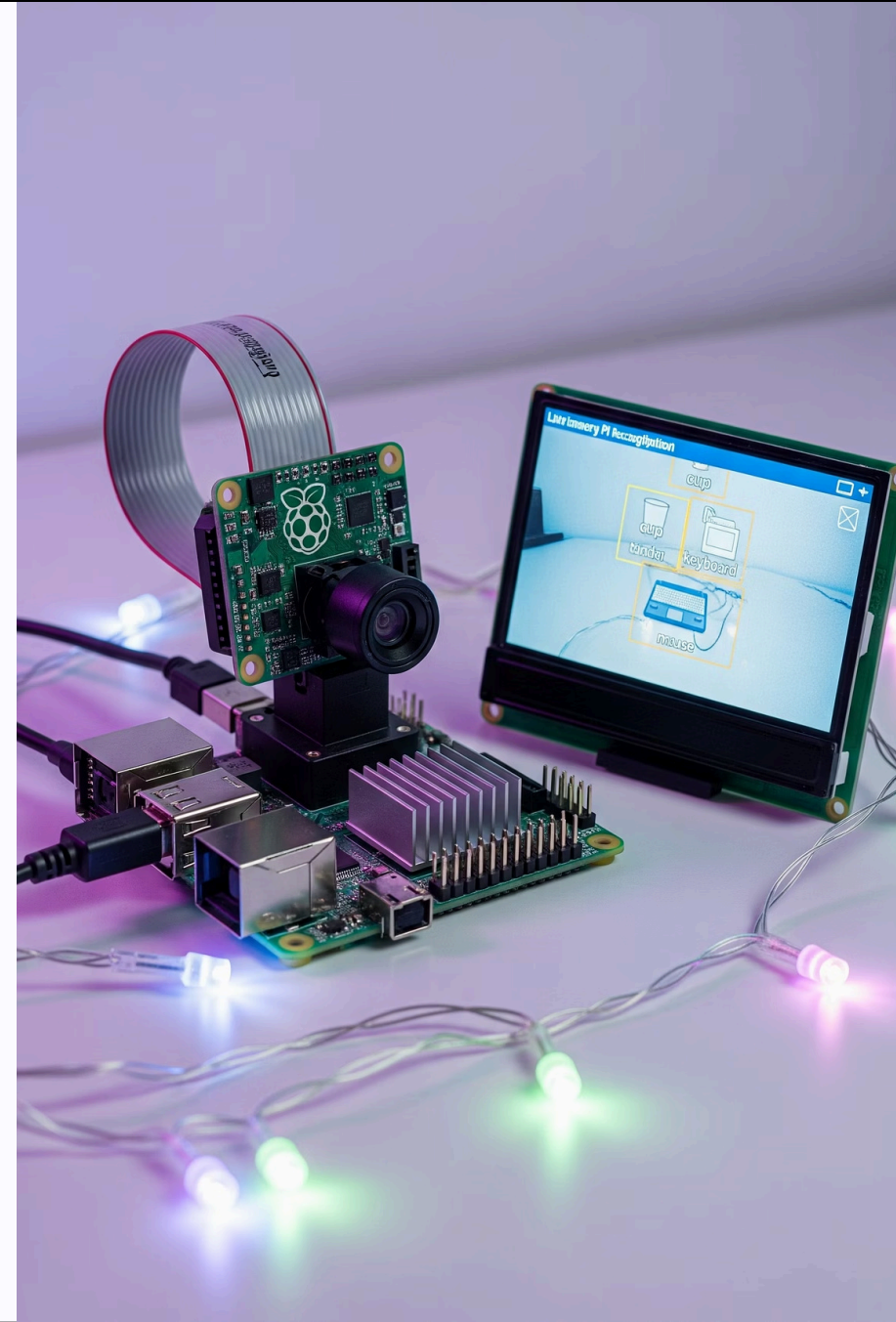


Running Vision Models on Raspberry Pi

Affordable edge AI with ARM-based devices for developers, makers and educators



Why Raspberry Pi for AI?

Low-cost, accessible

Affordable entry point to edge computing at \$35-75, democratizing AI development

Ideal for prototyping & education

Perfect balance of capability and simplicity for learning and iterating quickly

Camera interface built-in

Native CSI connector and USB support for various camera options

Broad community support

Extensive libraries, tutorials and forum assistance for AI development





Hardware Requirements



Raspberry Pi 4

4GB or 8GB RAM recommended for handling model memory requirements

ARM Cortex-A72 CPU provides necessary computing power



MicroSD Card

≥32GB, Class 10 speed for optimal performance

SanDisk Extreme or Samsung EVO recommended for reliability



Camera

Pi Camera Module v2 (8MP) or USB webcam

HQ Camera (12.3MP) for higher resolution applications



Optional: Coral TPU

USB Accelerator for 10-50× faster inference speeds

Enables real-time processing of more complex models

Software Stack

Operating System

- Raspberry Pi OS (64-bit recommended)
- Ubuntu 20.04 LTS also viable
- Enables full use of available RAM

Frameworks

- TensorFlow Lite - optimized for edge
- PyTorch Lite - for PyTorch users
- OpenCV - image processing pipeline

Optimizations

- ONNX Runtime ARM builds
- Coral EdgeTPU runtime
- ARM NEON acceleration

Languages

- Python 3 (primary choice)
- C++ (optional for performance)
- Shell scripts for automation



TensorFlow Lite Workflow

Train/export model

Use MobileNet, YOLO-lite or other lightweight architectures

Train on powerful machine, then export for deployment

Convert to .tflite format

Use TensorFlow Lite Converter

Apply quantization to reduce model size (INT8)

Deploy model to Pi

Transfer .tflite file and dependencies

Set up camera input pipeline

Run inference

Use TFLite interpreter for efficient execution

Process results for your application

```
import tflite_runtime.interpreter as tflite
import numpy as np
```

```
interpreter = tflite.Interpreter(
    model_path="mobilenet.tflite")
interpreter.allocate_tensors()
```

```
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
```

```
img = np.random.rand(1,224,224,3).astype(np.float32)
interpreter.set_tensor(input_details[0]['index'], img)
interpreter.invoke()
output = interpreter.get_tensor(
    output_details[0]['index'])
```

ONNX Runtime on Raspberry Pi



What is ONNX?

Open Neural Network Exchange format for interoperability between frameworks

Installation

```
pip install onnxruntime
```

Benefits

- Run models from any framework (TensorFlow, PyTorch, etc.)
- ARM-optimized builds for improved performance
- Memory-efficient inference on Pi's limited RAM
- Supports popular vision models (ResNet, MobileNet)

Acceleration with Coral EdgeTPU

10-50× speedup with USB accelerator

How it works

Dedicated neural processing unit optimized for TensorFlow Lite models

Handles matrix multiplications in hardware

Implementation

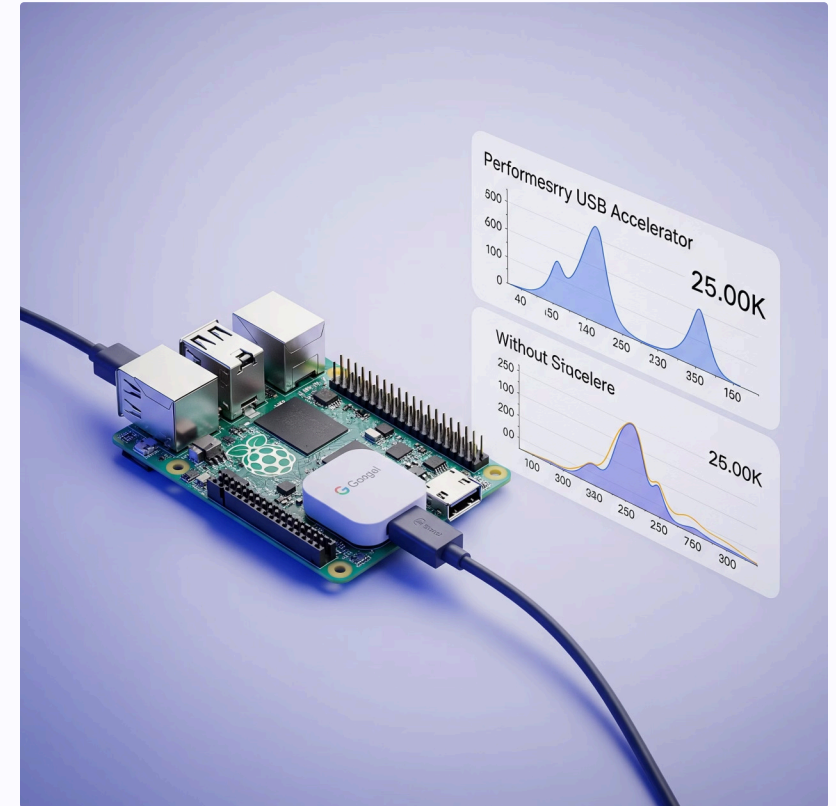
Compile .tflite model with EdgeTPU compiler

Load with specialized EdgeTPU runtime

Real Performance

MobileNet SSD detection at 30+ FPS on Pi 4

Makes real-time vision applications viable



Example Applications



Smart Doorbell

Detect and recognize visitors at your entrance

Send notifications with visitor images



Wildlife Monitoring

Motion-triggered species detection

Long-term battery operation in remote locations



Retail Analytics

Track shelf occupancy and product availability

Generate real-time inventory reports



Education

Low-cost AI robotics projects

Teach computer vision fundamentals

Challenges



Limited Resources

RAM (4-8GB) constrains model size and batch processing

CPU-bound performance without accelerators

Thermal Constraints

Heatsinks/fans required for sustained workloads

Potential thermal throttling under heavy load

Power Limitations

5-15W power consumption impacts portable applications

Battery operation requires careful power management

Processing Bottlenecks

Real-time inference only feasible with light models

Limited batch processing capability

Best Practices



Use Lightweight Architectures

- MobileNet v2/v3
- EfficientNet-Lite
- YOLO-Nano
- MobileViT



Apply Quantization

- Convert FP32 → INT8
- 3-4× smaller models
- 2-3× faster inference
- Minimal accuracy loss



Optimize I/O Pipeline

- Preprocess with OpenCV
- Resize images before inference
- Use threading for camera capture
- Implement frame skipping when needed



Consider Hardware Accelerators

- Coral TPU USB Accelerator
- Neural Compute Stick 2
- Google Edge TPU Dev Board
- Custom NPU HATs

Remember: Optimizing for the edge requires different thinking than cloud-based AI!

Key Takeaways



Raspberry Pi is your gateway to edge vision AI

Accessible, affordable, and supported by a vibrant community

Optimize aggressively for edge deployment

TFLite/ONNX with quantization is essential for good performance

Accelerate for production use cases

EdgeTPU or similar NPUs enable real-time processing

Perfect for specific use cases

Ideal for prototyping, education, and lightweight production deployments where cost and accessibility matter more than raw performance